Operating Systems

Sheet 8 Solutions

Memory Management

Name: Yehia Said Gewily

ID: 21010705

Date: April 2026

1 Memory Management Requirements (7.1)

1. Why is Relocation Desirable?

Relocation is important because a process may not always stay in the same memory location during execution.

- A process may be swapped out to disk and later returned to memory in a different place.
- The operating system may move processes during compaction to reduce fragmentation.
- Relocation allows the OS to use memory more efficiently, because a process can be loaded into any suitable free area.

So, relocation makes memory management more flexible and improves memory utilization.

2. Logical, Relative, and Physical Addresses

Type	Meaning
Logical	Address generated by the CPU and used by the program.
Relative	Address given as an offset from the beginning of the program or segment.
Physical	The real address in main memory (RAM).

Summary: The program works with logical or relative addresses, while the hardware finally accesses the physical address.

2 Memory Partitioning (7.2)

3. Internal vs. External Fragmentation

Internal Fragmentation:

This is wasted space **inside** an allocated block. It happens when a process is given more memory than it actually needs.

Example: A process needs 14K, but it is allocated 16K. The extra 2K is internal fragmentation.

External Fragmentation:

This is wasted space **between** allocated blocks. It happens when free memory is split into many small holes.

Example: Total free memory may be enough for a process, but it is not in one continuous block.

In short: Internal fragmentation is waste inside a block, while external fragmentation is waste between blocks.

4. Pointer Size in Fixed Partitioning

Given:

- Total memory $= 2^{24}$ bytes
- Partition size $= 2^{16}$ bytes

Number of partitions:

$$\frac{2^{24}}{2^{16}} = 2^8 = 256$$

To identify one partition out of 256, we need:

$$\log_2(256) = 8$$

Answer: $\boxed{\text{8 bits}}$

5. Worst-Fit: Pros and Cons

Worst-fit places a new process in the **largest** available free block.

Advantages:

- The remaining free block after allocation is usually still large.
- It may reduce the creation of very tiny holes.

Disadvantages:

- It uses up the largest block, which may be needed later for a big process.
- It usually requires searching the whole free list.
- In practice, it is often less efficient than first-fit.

Comparison:

- **First-fit:** fast and simple
- **Next-fit:** similar to first-fit but continues from last position
- **Best-fit:** chooses the smallest suitable block
- **Worst-fit:** chooses the largest suitable block

6. Dynamic Partitioning Placement

Free holes are:

$$20@20,\ 40@40,\ 20@140,\ 10@160,\ 40@230,\ 30@290,\ 40@360$$

The requests are: **40M, 20M, 10M**

a. First-fit

- 40M → start at 40
- 20M → start at 20
- 10M → start at 140

b. Best-fit

- 40M → start at 40
- 20M → start at 20
- 10M → start at 160

c. Next-fit (starting pointer at 0)

- 40M → start at 40
- 20M → start at 230
- 10M → start at 290

d. Worst-fit

- 40M → start at 40
- 20M → start at 230
- 10M → start at 360

7. Memory Configuration Analysis

a. Maximum size of the swapped-out process

The largest free block shown is 4M, so the maximum possible size of the swapped-out process is:

$$\boxed{4\text{M}}$$

b. Size of the free block before X was placed

Process X has size 2M, and to its right there is 3M free. So before placing X, the free block was:

$$2\text{M} + 3\text{M} = \boxed{5\text{M}}$$

c. New 3M request

Possible holes of size at least 3M are:

$$F_2 = 3\text{M},\ F_4 = 3\text{M},\ F_6 = 4\text{M},\ F_7 = 3\text{M}$$

- **Best-fit:** F_2
- **First-fit:** F_2
- **Next-fit:** F_2
- **Worst-fit:** F_6

8. Buddy System: 1 MB Block

a. Allocation sequence

Requests:

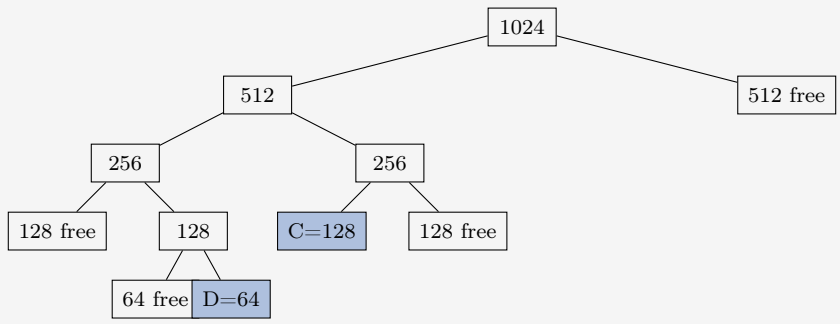
- 70K $\rightarrow$ 128K = A
- 35K $\rightarrow$ 64K = B
- 80K $\rightarrow$ 128K = C
- 60K $\rightarrow$ 64K = D

Memory states:

- Initial: $[1024K\ free]$
- After A: $[A = 128 \mid 128 \mid 256 \mid 512]$
- After B: $[A = 128 \mid B = 64 \mid 64 \mid 256 \mid 512]$
- After C: $[A = 128 \mid B = 64 \mid 64 \mid C = 128 \mid 128 \mid 512]$
- Return A: $[128 \mid B = 64 \mid 64 \mid C = 128 \mid 128 \mid 512]$

- After D: $[128 \mid B = 64 \mid D = 64 \mid C = 128 \mid 128 \mid 512]$
- Return B: $[128 \mid 64 \mid D = 64 \mid C = 128 \mid 128 \mid 512]$
- Return D: $[256 \mid C = 128 \mid 128 \mid 512]$
- Return C: $[1024K \ free]$

b. Binary tree after Return B



9. Buddy Addresses

Given block address:

`011011110000`

a. If block size $= 4 = 2^2$
Flip bit 2:

$\boxed{\texttt{011011110100}}$

b. If block size $= 16 = 2^4$
Flip bit 4:

$\boxed{\texttt{011011100000}}$

3 Paging (7.3)

10. Paging System Parameters

Given:

- Physical memory $= 2^{32}$ bytes
- Page size $= 2^{10}$ bytes
- Number of logical pages $= 2^{16}$

a. Bits in logical address

$$16 + 10 = \boxed{26 \text{ bits}}$$

b. Bytes in a frame

$$2^{10} = \boxed{1024 \text{ bytes}}$$

c. Bits for frame number in physical address

$$32 - 10 = \boxed{22 \text{ bits}}$$

d. Number of page table entries

$$\boxed{2^{16} = 65536}$$

e. Bits in each page table entry

$$\boxed{22 \text{ bits}}$$

11. Logical Address as (p, w)

If:

- a = logical address
- z = page size

Then:

$$\boxed{p = \left\lfloor \frac{a}{z} \right\rfloor} \qquad \boxed{w = a \bmod z}$$

where:

- p is the page number
- w is the offset inside the page

4 Segmentation (7.4)

12. Segment Address Translation

Segment table:

Segment	Start	Length
0	660	248
1	1752	422
2	222	198
3	996	604

Rule: if $d <$ length, then

$$\text{physical address} = \text{start} + d$$

otherwise, there is a segment fault.

a. (0,198)

$$660 + 198 = \boxed{858}$$

b. (2,156)

$$222 + 156 = \boxed{378}$$

c. (1,530)

$$530 \geq 422 \Rightarrow \boxed{\text{segment fault}}$$

d. (3,444)

$$996 + 444 = \boxed{1440}$$

e. (0,222)

$$660 + 222 = \boxed{882}$$

5 General Questions

13. 16 KB Memory: Placement Strategies

Sequence:

$$P1 = 7K, \quad P2 = 4K, \quad P1 \text{ terminates}, \quad P3 = 3K, \quad P4 = 6K$$

a. First-fit

- P1 at 0–7K
- P2 at 7–11K
- P1 leaves, free hole 0–7K
- P3 placed at 0–3K
- Remaining holes are 4K and 5K, so P4 cannot be loaded

Final map:

Range	Size	Status
0–3K	3K	P3
3–7K	4K	free
7–11K	4K	P2
11–16K	5K	free

Internal fragmentation $= 0$
External fragmentation $= 4K + 5K = 9K$

b. Best-fit

- P1 at 0–7K
- P2 at 7–11K
- P1 leaves
- P3 placed in the 5K hole at 11–14K
- P4 placed in the 7K hole at 0–6K

Final map:

Range	Size	Status
0–6K	6K	P4
6–7K	1K	free
7–11K	4K	P2
11–14K	3K	P3
14–16K	2K	free

Internal fragmentation $= 0$
External fragmentation $= 1K + 2K = 3K$

c. Buddy System

- P1 (7K) rounds to 8K
- P2 (4K) rounds to 4K
- P1 leaves
- P3 (3K) rounds to 4K
- P4 (6K) rounds to 8K but cannot be loaded

Final map:

Range	Size	Status
0–4K	4K	P3
4–8K	4K	free
8–12K	4K	P2
12–16K	4K	free

Internal fragmentation = 1K
External fragmentation = 8K free but not mergeable for P4

d. Simple Paging (page size = 2K)

- Total frames = $16/2 = 8$
- P1 (7K) needs 4 pages
- P2 (4K) needs 2 pages
- P1 leaves
- P3 (3K) needs 2 pages
- P4 (6K) needs 3 pages

All processes can be loaded.

Possible final frame map:

Frame	Range	Owner
0	0–2K	P3
1	2–4K	P3
2	4–6K	P4
3	6–8K	P4
4	8–10K	P2
5	10–12K	P2
6	12–14K	P4
7	14–16K	free

Internal fragmentation = 1K
External fragmentation = 0

Summary

Strategy	P4 loaded?	Internal frag.	External frag.
First-fit	No	0	9K
Best-fit	Yes	0	3K
Buddy	No	1K	8K
Paging	Yes	1K	0